Eurocontrol PRC data challenge 2026: Predicting taxi-out time

Methodology report and results.
lighthearted-sandwich team

Document written by Bruno Serio with the assistance of AI (qwen3.8-flash-next served via the Unsloth Studio API running on a local machine), translated by Deepl and proofread by Amanda Martinez Gil

Faculty member, Master's Programme in Aeronautics, Transport and Energy Engineering (CMI ATE)
University of Paris Nanterre

13 October 2026

Table of Contents

Executive summary 3
Key results 4
1. Data and scope of the competition 5
1.1 Data file naming conventions 5
1.2 Data file size and competition submission 5
1.3 Overview of the ten airports within the study area 6
1.4 Analysis of flight data files 7
2. Experimental analysis of the data 9
2.1 Distribution of the target to be predicted (, taxi-out) 9
2.2 Identification of missing data 10
2.3 Daily cycle: first signal 11
2.4 Season cycle: second signal 12
2.5 Anomalies: physically implausible values 13
2.6 Summary of the data analysis for modelling 14
3. Data engineering under causal constraints 14
3.1 Choosing the timing of the prediction 14
3.2 The 34 parameters of the submitted specification 16
3.3 Cost of restrictions 18
3.4 Data tested and subsequently rejected 18
4. Validation protocol 19

4.1 Checking for information leakage 19
4.2 Time reserve: principle and use 19
4.3 Adoption threshold: determination and use 20
4.4 Links between the 2025 measurements and the 2026 ranking 20
5. Modelling and submitted solution 21
5.1 Single LightGBM model: 278 decision trees 21
5.2 Model training: metadata configuration 21
5.3 Locked setting recipe 22
6. Official results and critical analysis 22
6.1 Score used and forecast prior to submission 22
6.2 Breakdown by month and by airport 23
6.3 Rejected variants 25
7. Risk of data leakage: methodological dilemma and limitations of our hypothesis 25
7.1 Two interpretations of the verb ‘predict’ 25
7.2 Our trajectory, v1 to v7 26
7.3 Causal assumptions: conditions of validity and consequences of their violation 27
7.4 Recommendations to the organizers 28
8. Control via a free, local AI 28
8.1 The tooled chain 28
8.2 Why these two models, and why run them locally 29
8.3 The balance of things: what the AI did not decide 29
9. Reproducibility, acknowledged limitations and conclusion 30
9.1 Obtaining the data, not redistributed 30
9.2 Step-by-step reproduction 30
9.3 What was actually submitted 30
9.4 Our limitations, without compromise 31
9.5 Conclusion 31
Annex — Open-source stack, AI tooling and licence compliance 33
A.1 Python stack (versions pinned as recorded on 13 October 2026) 33
A.2 AI orchestration tooling (all open source) 33
A.3 Language models (open weights, run locally) 34
A.4 License compliance statement 34
References 35

Executive summary

The problem: using 12 monthly files from 2025 containing 4.17 million lines corresponding to arrivals or departures (50/50), we need to predict the aircraft's taxi-out time, that is, the time elapsed between when an aircraft leaves its stand (pushback, or off-block) and when it leaves the runway on take-off (mvt_time_utc_mvt), for flights departing from ten major European airports , in 2026. The form to be completed, containing the data for evaluating the model, covers four months of 2026 – January, February, June and July – and the ranking focuses on January and July.

The central idea is that the test dataset (2026) masks the pushback time, but it contains the actual take-off time and a pushback time recorded by another source, the EUROCONTROL network manager (index _flt). The difference between these two times mimics the response to be predicted; this is a shortcut we refer to as the proxy.

The catch: both the training data (2025) and the test data (2026) therefore contain fields that provide access to the model's target (taxi-out). By using them, we achieve a better score (384 s), but we are no longer making a prediction ; we are reconstructing the past based on the fields provided by the network operator, which are data from after the flight in question.

Our choice : to use only the fields known before the aircraft moves. Our score (595 s) results directly from this constraint. We found no information in the regulations regarding whether or not this proxy could be used, and we explored both approaches before opting solely for the causal approach – that is, the one that excludes the use of the proxy.

The method: a single, explainable model (LightGBM), comprising 278 decision trees constructed successively, each correcting the errors of the previous ones. The final prediction is the sum of the terminal nodes reached. A strict validation protocol with a time limit and a quantified acceptance threshold, was selected. Finally, the team makes the decisions, and a free, local AI carries them out.

The challenge: a handful of anomalous data points, concentrated at Rome Fiumicino (LIRF), account for the majority of prediction errors (detailed measurements in section 6.4).

Managing the AI: we have integrated generative artificial intelligence into our workflow, but according to a specific philosophy. We are aware that AI currently surpasses us in certain capabilities—particularly cognitive ones—such as analysing vast amounts of data or translating complex reasoning into code. Rather than accepting this relegation by blindly delegating to cutting-edge commercial tools, we have chosen to deploy LLMs on local machines; whilst they are less powerful, they are open-source and reproducible.

Opencode [2] is responsible for breaking the problem down into tasks and for orchestrating the agents and tools required for the solution (Arbor AI). Our role involved first drafting the

instructions (prompts), but also conducting a rigorous critique of the results. The final decision rested with us. To remain in control of our tools, we adopt the **A-2L-R** approach:

- **Free & Local:** we use models with *open weights* run locally on our hardware (Mac Studio VRAM, NVIDIA GX10 GPUs, GPU-based graphics cards), such as **DeepSeekV4-flash-0731** [4] and **Qwen 3.8-flash-next** [5].
- **Reproducible:** we reject proprietary 'black boxes' to ensure that our approach can be understood, audited and reproduced without commercial constraints.

AI has assisted us in generating the code, drafting analysis summaries and carrying out benchmarks, but the code is the result of decisions we made during the various phases of AI management.

The team: the lighthearted-sandwich team comprises lecturers and students from the Master's programme in Aeronautical, Transport and Energy Engineering at the University of Paris Nanterre. Bruno Serio, Director of the CMI ATE, oversaw the design and technical development. Isaak Bathily (4th-year student), the challenge coordinator, and Adam Mecellem (3rd-year student) took part in the strategic discussions and the evaluation of submissions. Amanda Martinez Gil, a lecturer in the CMI management team, ensured the validation of the analyses produced by the students. Our shared ambition was to approach an aeronautical machine learning competition with the assistance of agent-based AI in a coherent, transparent and intellectually honest manner.

Key results

Submitted version	Official score(s)	Type	Nature
v1; ..; v5	737; 599; 526; 521; 545	Causal / hybrid	Recipes discarded during testing
v6	384	Reconstruction	Recipe using the proxy
v7	**595**	**Causal prediction submitted**	**Locked setting recipe described in section 5**

Our local measurement of the submitted version (v7) for January and July is 603.5 s (reports/causal-r2.json) according to our metric; our official score is 595 s. The difference between our score for the causal v7 and the 384-second reconstruction (which takes the proxy into account) gives an indication of the margin allowed for the causal constraint: 211 seconds in our case.

The two files submitted to the organisers are named after the team: `lighthearted-sandwich_v7.parquet` (344,841 lines, January and July, score as above) and the single final submission `lighthearted-sandwich_final.parquet` (670,790 lines, four months). The source code is publicly available under the GNU GPLv3 licence (reference to be added).

1. Data and scope of the competition

1.1 Data file naming conventions

The data distributed by the organisers (in Parquet format) is the result of a reconciliation between the movements reported by airports (columns suffixed with `_mvt`) and the calculations of the EUROCONTROL network manager (columns suffixed with `_flt`). Each row represents an aircraft movement identified by `MVT_ID_mvt`, either a departure (`PHASE_mvt = "DEP"`) or an arrival (`PHASE_mvt = "ARR"`). Only departures are to be predicted.

Pushback (off block) is the moment when the aircraft leaves its parking position (pushed backwards by a tractor) and thus begins to taxi under its own power . In the competition data, as in EUROCONTROL practice, the timestamp of this first movement serves as the start of the taxi-out time. This is also why we have chosen this as our prediction point. Indeed, at this precise moment, the aircraft's departure becomes a certainty. **Take-off** (`MVT_TIME_UTC_mvt`) is the moment when it leaves the runway. The **taxi-out time** (`TAXITIME_SEC_mvt`) is the interval between the two points in time (take-off and pushback), expressed in seconds; this is the model's target. The **RMSE** is the root mean square error, expressed in seconds, a metric that heavily penalises large errors. Network timetables (source: network operator) provide a framework for pushback: estimated (`EOBT_1_flt`), initially planned (`IOBT_flt`), last known value (`LOBT_flt`) and actual (`AOBT_3_flt`). The time at which each of these fields is written to the system determines whether we are permitted to use it to guarantee a causal prediction, section 3.1.

1.2 Data file size and competition submission

The organiser has made available twelve-monthly training files (2025) and two pairs of test files (2026). The table below lists the files, and Figure 1 shows the monthly distribution of the training data.

File	Role	Rows
`training_2025-01-01…2025-12-01` (12 files)	learning history	4167797 including 2,085,047 departures and 2,082,750 arrivals
`ranking.parquet`	Variables for the months under review (January and July 2026) : departures and arrivals	689,534 (departures and arrivals)
`submitting.parquet`	template to be completed for the ranking (January and July)	344,841 (departures)
`final_ranking.parquet`	Variables for the four months of the final response	1,341,197 (departures and arrivals)

File	Role	Rows
`final_submitting.parquet`	template for the single final response, named `lighthearted-sandwich_final.parquet`	670,790 (departures)

Submissions must be made via the MinIO client only, with a maximum of five per day. The ranking will consider the team's best submission from January and July [6]. The single final submission must be submitted by 12:00 on 14 October 2026 and cover the four months specified in the form.

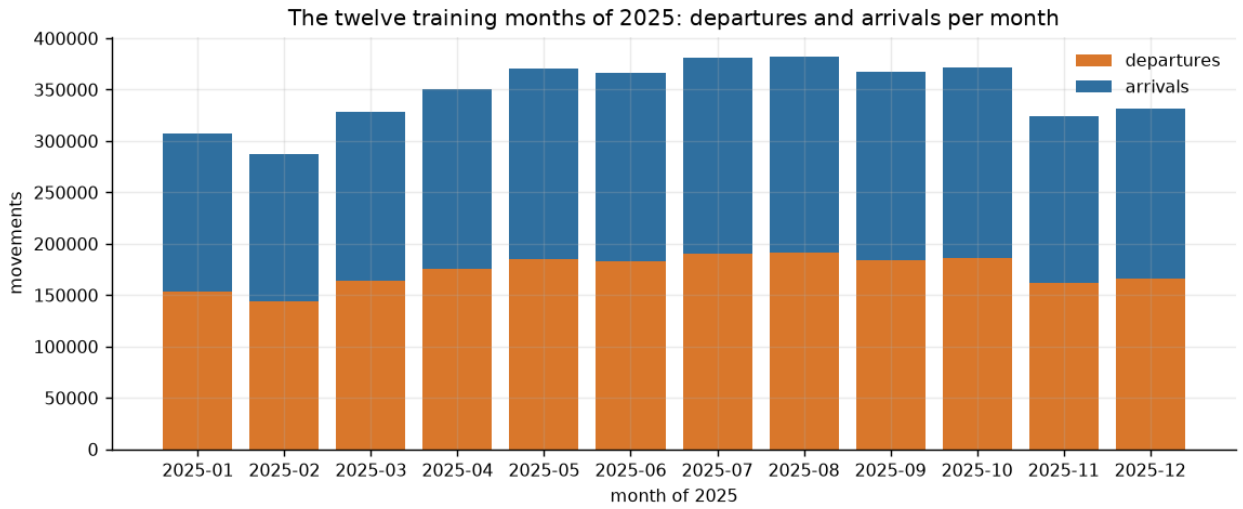


Figure 1: Monthly distribution of transactions

Volume remains stable at around 340,000 to 380,000 transactions per month (Figure 1), with a winter trough in February and a summer peak in July–August. Departures and arrivals are roughly balanced at 50/50. The analysis therefore covers both seasonal patterns for the periods in question – January in winter and July in summer – and no specific weighting of particular months was warranted.

1.3 Overview of the ten airports within the study area

The study covers ten major European airports: Istanbul (LTFM), Paris CDG (LFPG), London Heathrow (EGLL), Amsterdam (EHAM), Madrid (LEMD), Frankfurt (EDDF), Barcelona (LEBL), Rome Fiumicino (LIRF), Munich (EDDM) and Zurich (LSZH).

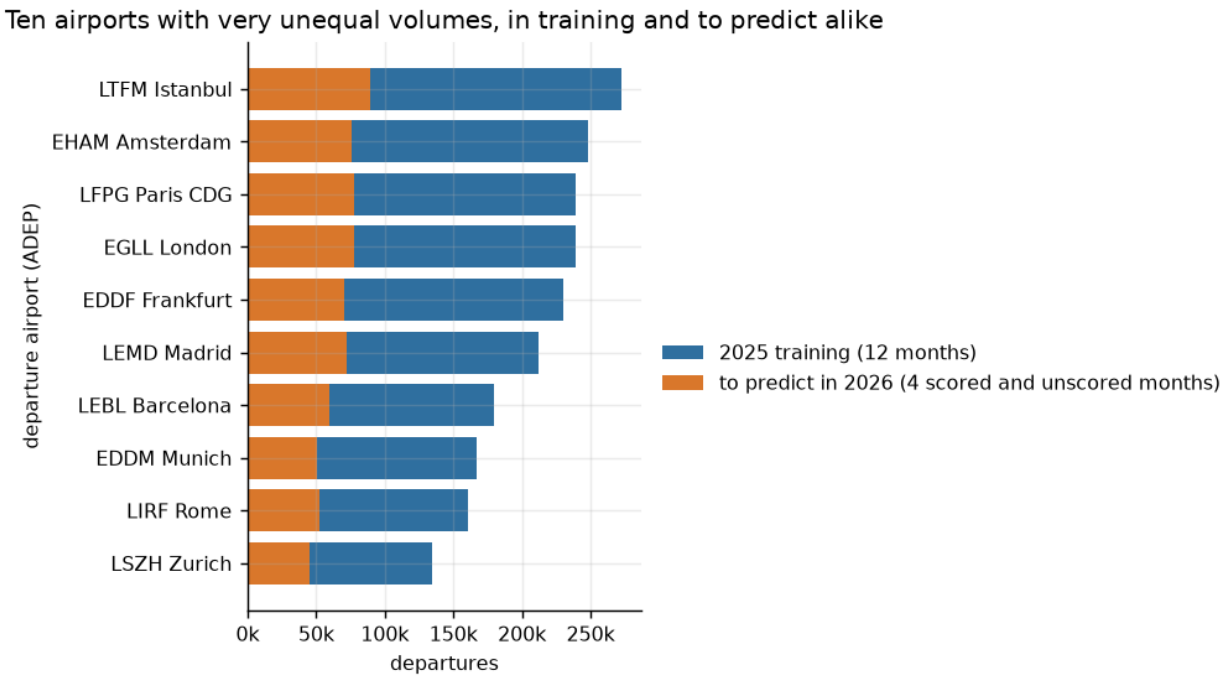


Figure 2: Departures by airport (year 2025)

Volumes (Figure 2) vary by a factor of two depending on the airport: 273,000 departures in Istanbul compared with 135,000 in Zurich during the training year. This hierarchy is reflected in the population to be predicted. The implication for modelling is that a global model must account for significant heterogeneity amongst the 10 airports in the study; 6 out of 34 parameters are sufficient to ensure this, all indexed to the airport, as detailed in section 3.2. Six of the 34 parameters are built from the airport, in addition to the departure-airport identity itself : 3 smoothed taxiing averages: by stand (te_ap_stand); by hour (te_ap_hour); by day and slot (te_ap_dow_slot). And 3 expected taxi distances: from the allocated stand (geo_stand_sec), from the expected runway section (geo_exp_rwy_sec); and the sum of the two (geo_exp_total_sec). 'Smoothed' here means adjusted for group size. None of these six are columns in the provided files: they are mapping tables, constructed during training by *app/src/features.py*, written once and for all in *models/causal/spec.json*, and read back as they are during inference by *app/infere.py*.

1.4 Analysis of flight data files

Three structural facts determined our entire approach, all three of which can be verified by analysing the parameters of the official files.

a) The target is the result of subtracting two data points from the flight history: for the 2,085,047 departures in 2025, `TAXITIME_SEC_mvt` equals exactly `MVT_TIME_UTC_mvt` minus `BLOCK_TIME_UTC_mvt`, matching to the nearest second on 100 per cent of the rows (see Figure 3).

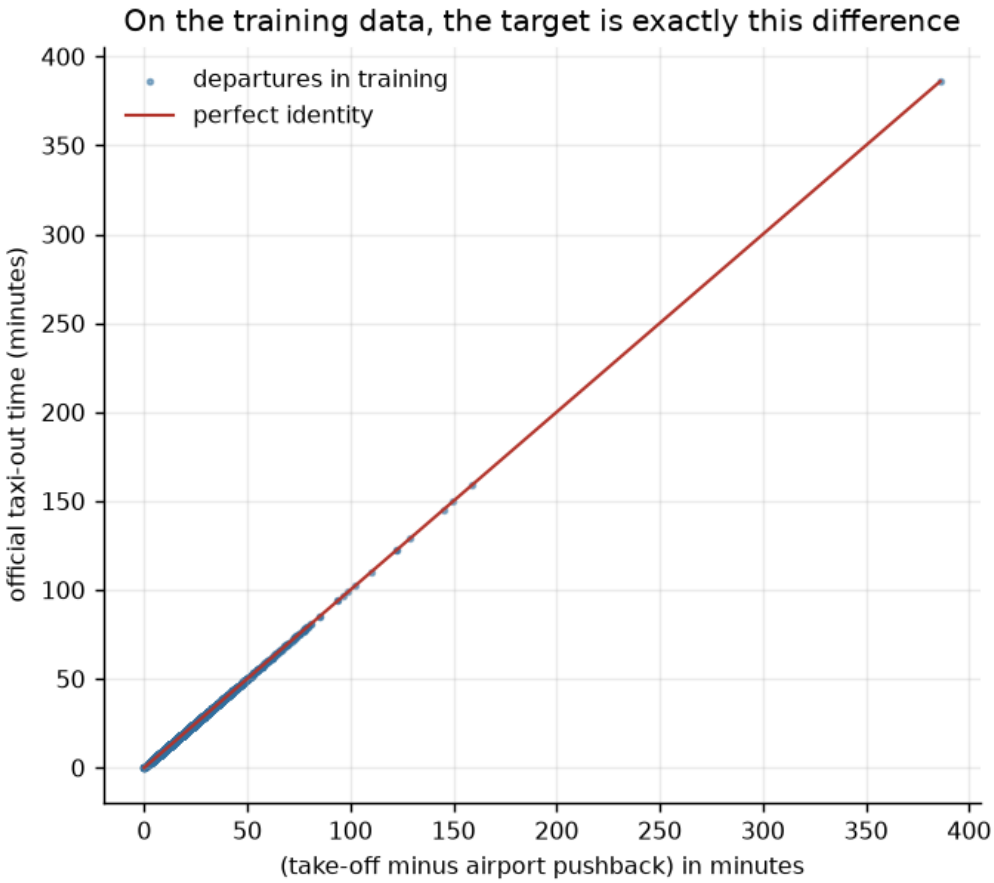


Figure 3: Target consistency: [take-off time – pushback time] vs [official taxi-out time]

The points in Figure 3 lie on a straight line (y = x), confirming the definition of the target across all the points in the training files.

b) The test masks one hour in every two: of the 670,790 departures to be predicted, `BLOCK_TIME_UTC_mvt` and `TAXITIME_SEC_mvt` are missing from 100 per cent of the rows, so direct subtraction is therefore impossible. However, the take-off time `MVT_TIME_UTC_mvt` is present in 100 per cent of the rows. We have verified that this is not a scheduled time; the published timetable and this column coincide in only 0.01 per cent of the rows, and its median deviation from the timetable reaches 1,440 seconds, or twenty-four minutes. This is indeed an actual time, unknown at the time of prediction in practice, but clearly present in the file.

c) A third column enables the taxi-out time to be calculated: the pushback time recorded by the network manager, `AOBT_3_flt`, is present for 98.7 per cent of the departures to be predicted. Subtracting this from the actual take-off time approximates the target with a measured RMSE of approximately 385 seconds without any training. The answer is therefore almost written into the test file, and our v6 officially calculated it as 384.16 s before we definitively closed that door for good, considering it preferable to predict the taxi-out time.

These three facts form the starting premise of the entire report. They underpin the distinction between predicting and reconstructing, which is discussed in section 3.1 and then in section 7.

2. Experimental analysis of the data

Before any modelling took place, we systematically analysed the 2025 historical data. Each sub-section below answers a simple question, and each answer has guided a design choice that has since been verified. The figures are taken from the twelve official files, covering 2,085,047 departures.

2.1 Distribution of the target to be predicted (, taxi-out)

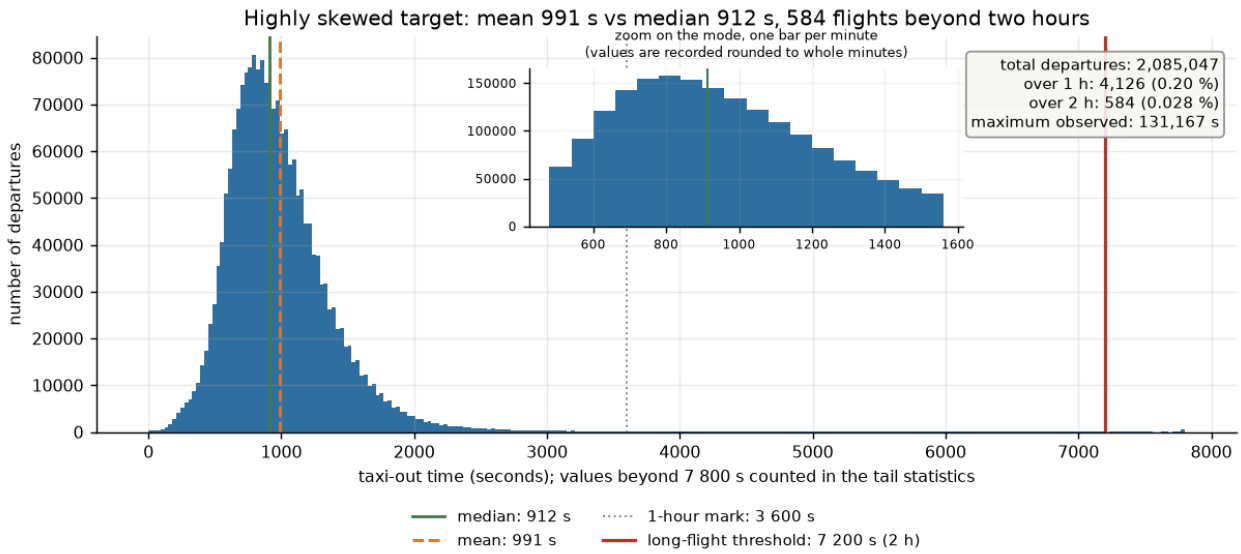


Figure 4: Distribution of taxiing time

The distribution (Figure 4) is asymmetrical with a long tail on the right. The median is 912 s, the mean 991 s, and the central interval ranges from 480 to 1,684 s, or eight to twenty-eight minutes. 4,126 departures exceed 1 hour and 584 exceed 2 hours. Consequently, the majority of flights fall within a narrow range that any reasonable benchmark already comes close to achieving, and the bulk of the variation lies in the tails, which the RMSE penalises squared.

2.2 Identification of missing data

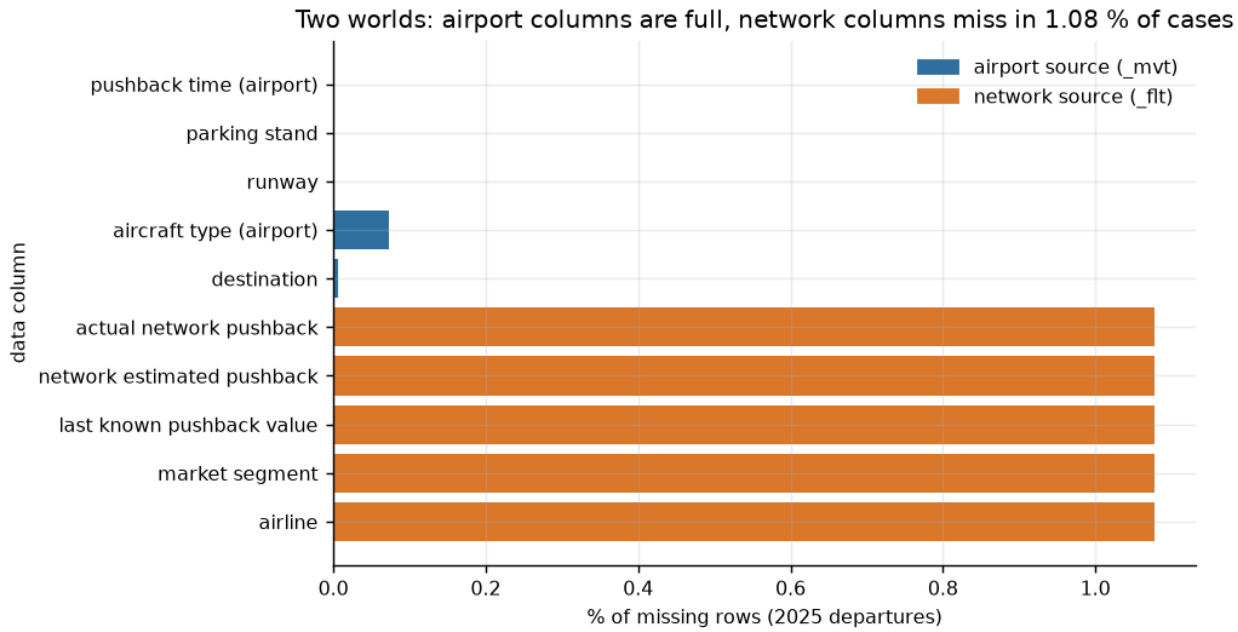


Figure 5: Proportion of missing data

The columns reported by the airports are virtually complete; the parking bay and airport pushback time are never missing. The columns provided by the network manager (_flt), on the other hand, are collectively missing in exactly 1.08 per cent of departures; this rate is identical across all columns because they all disappear at the same time when the flight has no network record. Consequently, a variable to track the 'absence of a network record' has been adopted as a control variable for missing parameters in the model.

2.3 Daily cycle: first signal

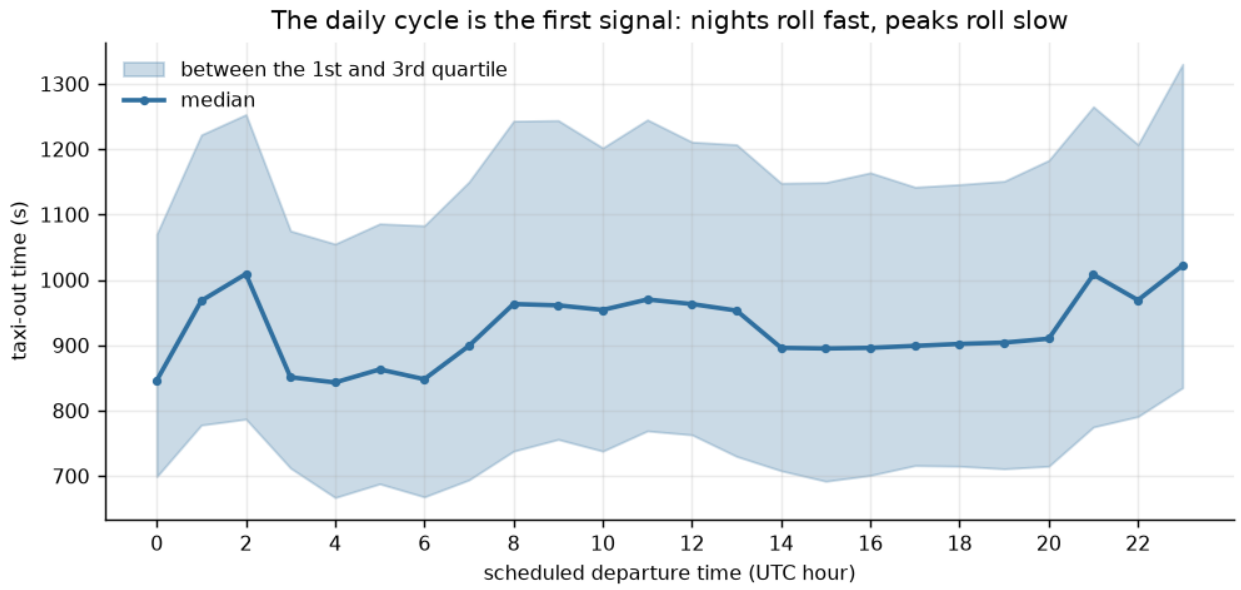


Figure 6: Hourly cycle: taxiing vs. departure time

At night, aircraft travel faster; during the morning and evening rush hours, they travel much more slowly; and dispersion increases during the day. The scheduled time is therefore the primary determinant of the target; it forms the basis of our internal reference and justifies the cyclical encoding of the time in the model's variables.

2.4 Season cycle: second signal

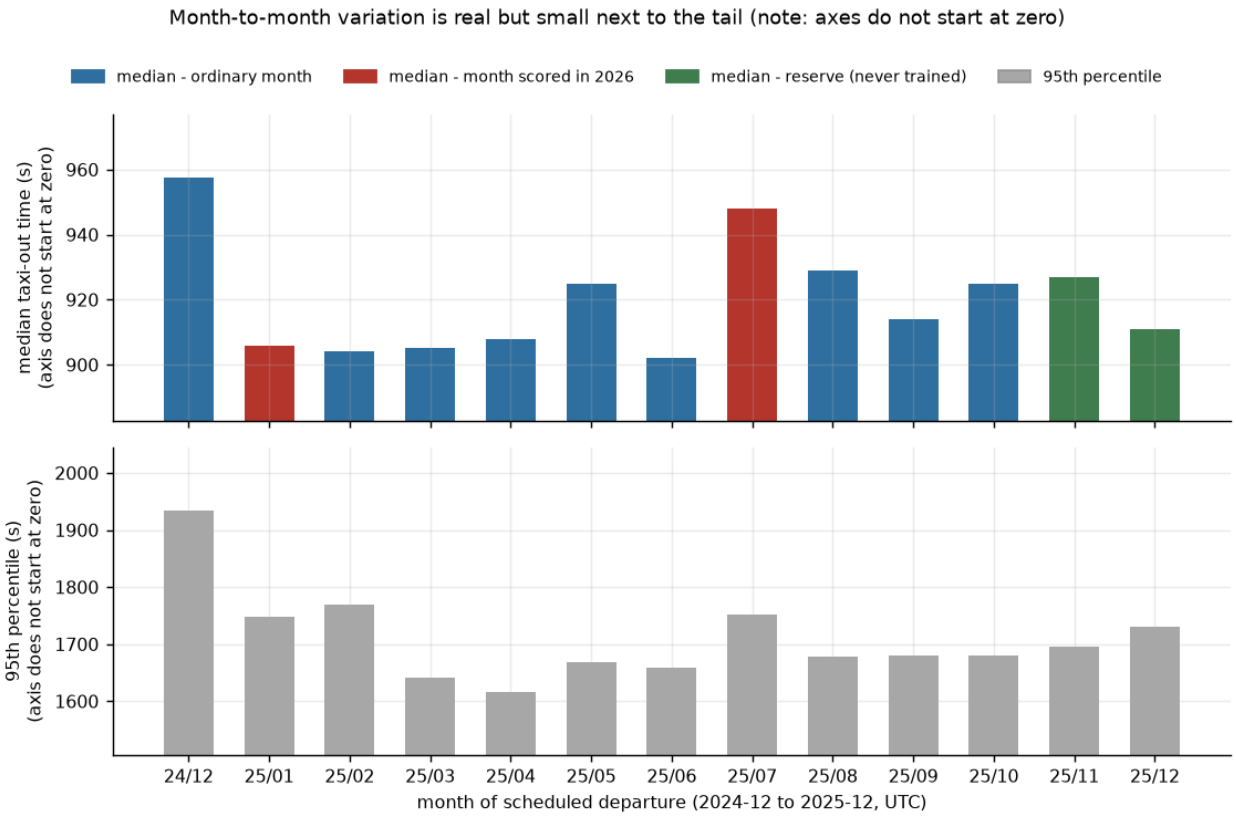


Figure 7: Seasonality: monthly distribution of taxiing time

The average driving time varies from month to month (Figure 7); July is significantly more challenging than June, with the same median but a much heavier tail. The two months marked in 2026—January in red and July in red—therefore require separate assessment rather than an average; a model judged solely on the average of the two would mask precisely what is causing the problem. The green months, November and December, were never used for training; their role is explained in section 4.2.

2.5 Anomalies: physically implausible values

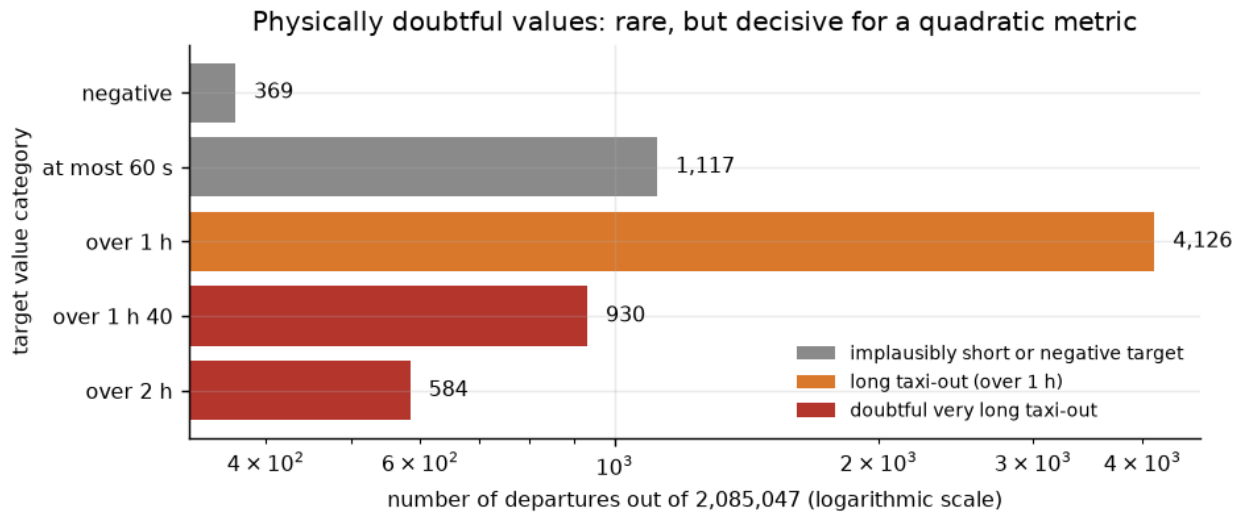


Figure 8: Outliers in driving time

369 departures show a negative journey time (Figure 8), which is physically impossible; 1,117 are less than one minute; 930 take 1 hour 40 minutes; and 584 exceed two hours. These last entries represent fewer than one in a thousand, but account for more than half the sum of the squares of the errors (target RMSE) of any of the models we trained to predict them. To distinguish between measurement error and extreme reality, we compared the two independent sources of pushback data: that from the airport and that from the network operator (_mvt and _flt).

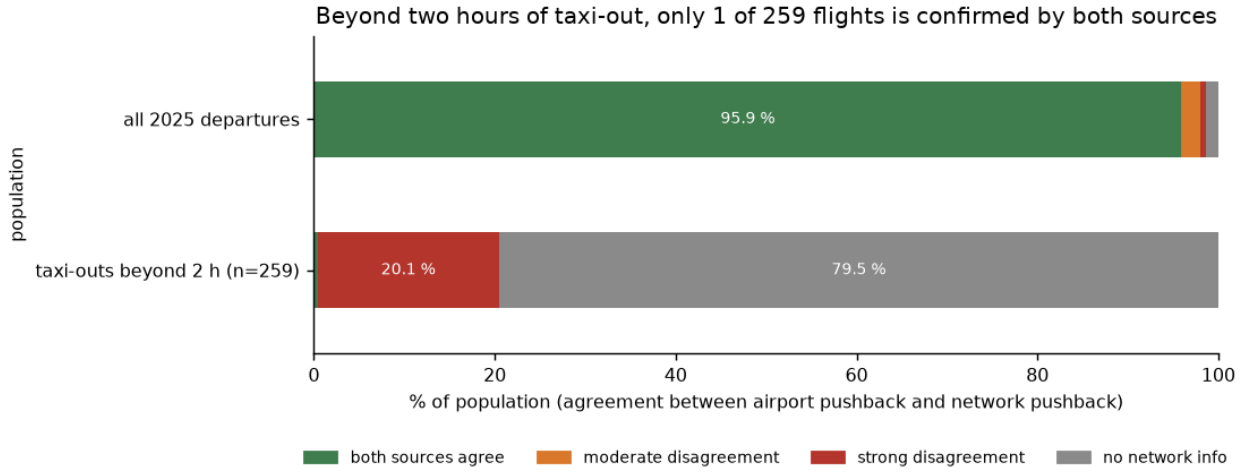


Figure 9: Breakdown of discrepancies between sources (_mvt, _flt)

Across all departures, the two sources (Figure 9) are largely consistent. Beyond two hours of reported taxiing time, almost no flights are confirmed by both measurements – approximately one in 259 in our audit. These outliers are therefore not corroborated by a second measurement. This has two consequences: we refuse to treat them as of the truth,

without, however, excluding them or scaling back our predictions, as every alternative processing method has been found to be inferior (section 6.4). They will remain the irreducible part of the score, particularly in Rome (LIRF), where the proportion of doubtful records is 20 times higher than normal.

2.6 Summary of the data analysis for modelling

From the previous analysis, we draw the following conclusions, which are useful prior to any training. A quadratic metric and unconfirmed Gaussian tails mean that the tail must be retained as it is and each idea assessed on a month-by-month basis. The heterogeneity of airports necessitates specific statistical benchmarks for each one. The dominant daily cycle means that the time of day serves as the backbone of the variables. The absence of network records necessitates the use of absence indicators. And the three structural facts described in section 1.4 show that the answer is virtually written into the test file. This situation necessitates a rule governing access to information (causal versus non-causal), which is the subject of the following section.

3. Data engineering under causal constraints

3.1 Choosing the timing of the prediction

The competition rules require **a prediction** to be made, but they never specify at what point the prediction must be made, nor do they provide a list of prohibited columns. Taken literally, everything contained in the files would therefore be usable, including information that only exists after the flight has ended, such as the actual take-off time. We have deliberately made a restricted choice by refusing to use the proxy to enter the competition. The rules are silent on several other points: the timing of the prediction – whether before pushback (to predict taxiing time), at take-off (to reconstruct lost data), or retrospectively (to reconstruct lost data) – and which columns are actually prohibited. We considered that the silence of the rules does not automatically constitute permission to use the data. We felt we did not have the right to use all the variables present in the files, for example `AOBT_3_flt` – the pushback time recorded by the network manager – which would provide the target with little or no need for learning. In this case, we are no longer answering the question posed – 'will this aeroplane fly for a long time?' – but another: 'how long has it flown?', which falls under reconstruction rather than prediction. Reporting the weather after the storm provides a perfect forecast. This is not a good prediction; it is a report. The rules did not prohibit a report; we chose to make a prediction.

We therefore set ourselves a stricter rule than that laid down in the text: all information used must be known before pushback, that is to say, before the aeroplane begins to move. It is against this rule that we judge our submission, and not merely against the minimum required by the regulations. The choice of this point in time is by no means arbitrary; a model based on pushback can in fact serve as a pre-departure operational tool for an airport manager.

This choice comes at a quantifiable cost – +211 seconds on the official score – and we accept that.

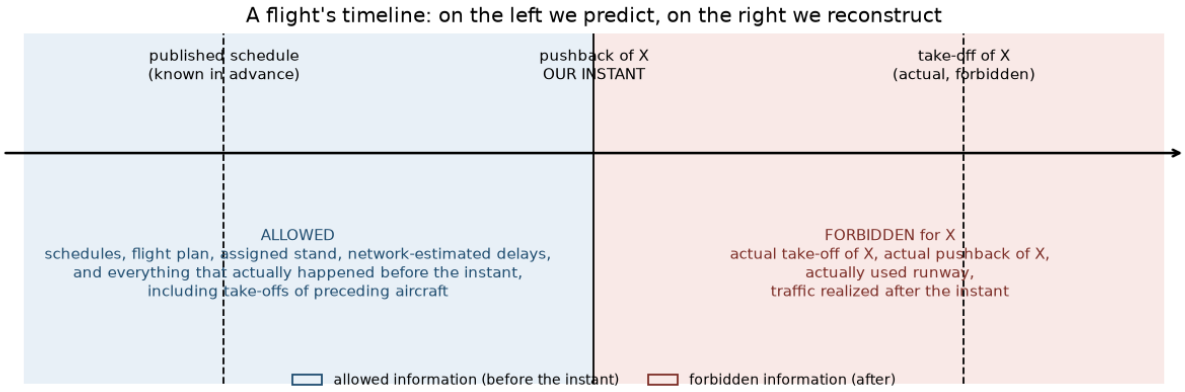


Figure 10: Timeline of authorised (blue) and prohibited (pink) data

The boundary (Figure 10) between permitted and prohibited data is not a list of columns; it is a point in time. To the left of this point in time, everything is permitted, including the actual observed history of other aircraft; take-offs from previous flights form part of the permissible information, particularly for measuring runway congestion at the airport. To the right, anything relating to the predicted aircraft is prohibited, starting with its actual take-off, as knowing this is equivalent to obtaining the answer by subtraction. As a practical point, the lines in the files are not arranged in the order of take-offs, and the model never looks at the neighbouring line; it operates in time windows, with the physical order of the queue being reconstructed from the timestamps.

This limitation has a corollary that we also apply to our own tests: traffic actually observed prior to the moment in question is permissible; we have tested this in practice, not merely in theory. Nine variables relating to take-offs and landings that had already occurred were incorporated into the causal track, `in the 'refaites-r1'` experiment from the register. They accounted for approximately 11 per cent of the model's decisions and contributed only 0.32 per cent to the overall RMSE, with January showing a deterioration of 0.87 per cent. As they fell below our adoption threshold (section 4.3), they were not included in the final model. The forecast traffic used by the model captures the bulk of the signal, with two advantages: it can be calculated well in advance and does not depend on any real-time data streams.

There remains one point that our analysis does not prove; we set this out, along with the relevant measurements, in section 7.3. The model reads three hours' worth of pushbacks scheduled by the network manager and the allocated stand, assuming this information is recorded before the aircraft taxis. The six variables derived from these fields are worth noting, as the discussion centres on them.

Model variable	Plain-language definition
d_eobt_min	Estimated pushback time by the network manager minus the published timetable, in minutes. A flight published for 14:00 with an estimate of 14:25 is +25
miss_EOBT_1_flt	equals 1 if the network has no record for this flight, 0 otherwise
d_iobt_min	Initially planned pushback time, set at the planning stage, minus the published schedule
miss_IOBT_flt	absence indicator for the initial value
d_lobt_min	latest known pushback time from the network minus the published timetable
miss_LOBT_flt	absence indicator for the last known value

3.2 The 34 parameters of the submitted specification

All are available prior to deployment, and each is traceable in the frozen model specification, the `models/causal/spec.json` file in the public repository, which records the exact list of variables and construction flags. This is the source to be checked to challenge or reproduce our compliance.

Family	Content	Why it is permitted
Scheduled departure timetable	time, day of the week, date, weekend, two cyclic time coordinates	taken from the published timetable, known months in advance
Flight details	departure airport, aircraft type (as declared in the movement and flight plan), winter and summer schedule category, market segment, IFR or VFR flight rules, flight type	Planning information
Delays already anticipated by the network	the six variables from the previous table	Fields calculated by the network manager prior to taxiing; design assumptions as specified in section 7.3
Expected ground geometry	taxi distance from the allocated stand, expected runway section, total, based on 2025 historical averages by airport, stand and time	The stand is allocated during planning; the distances are based on past experience, not on the current flight
Statistical benchmarks derived from 2025 data	Smoothed average taxi times by airport and stand, airport and time, airport, day and slot, aircraft type, operator	averages calculated using training months only, with open smoothing against small groups

Family	Content	Why it is permitted
Traffic scheduled around the departure	Expected arrivals in the previous, current and next slots; load factor relative to observed capacity; expected departures in the current slot	figures based on published timetables, never on actual traffic

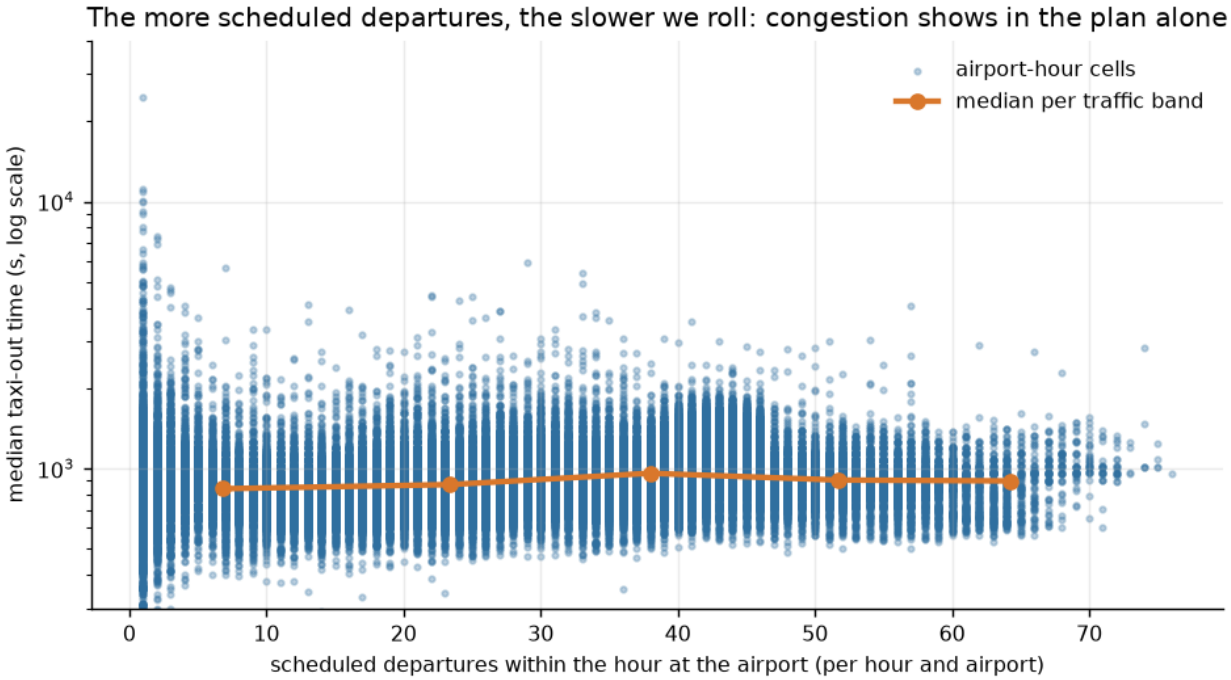


Figure 11: Planned congestion

Each point in Figure 11 represents an airport-hour pair for the year 2025. The trend is clearly rising; only the count of scheduled departures provides real information on taxiing time. This is the measured justification for our decision to rely on planned rather than actual traffic.

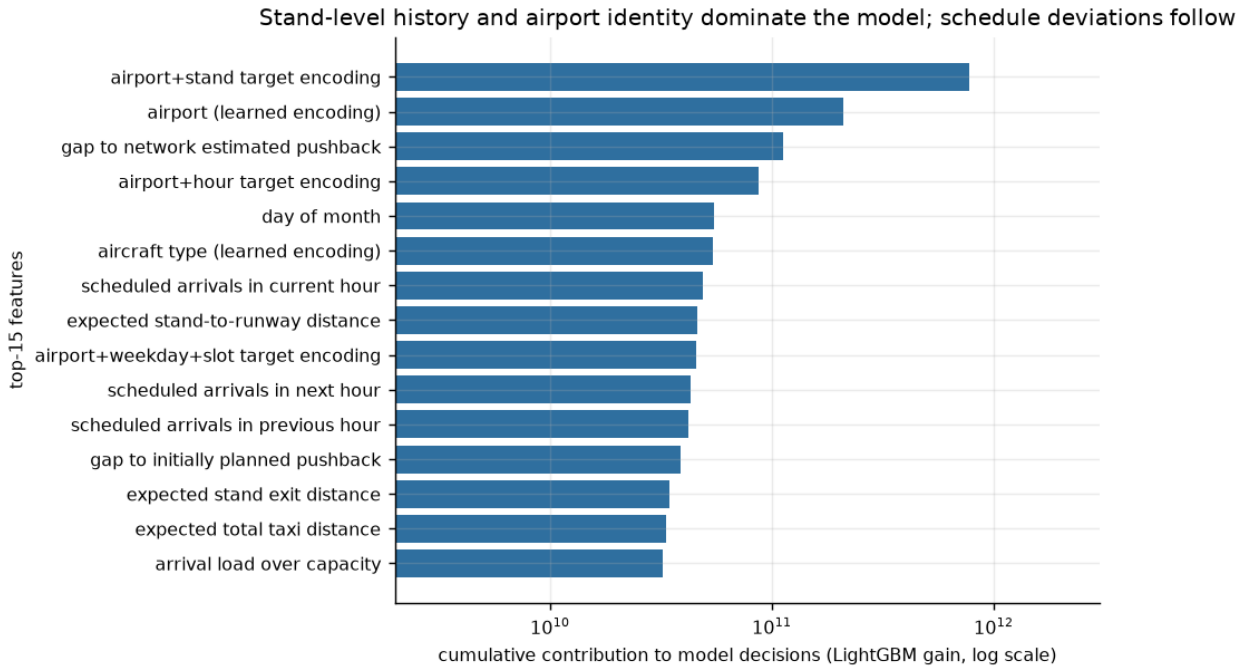


Figure 12: Importance of variables (Weights in the model)

Deviations from network timetables, expected geometry and statistical benchmarks are the key factors driving decisions. It is worth noting, with regard to section 7.3, that no single variable underpins the model – which would render any hypothesis overly fragile and therefore dangerous –; rather, the prediction is based on a converging set of families of independent parameters.

3.3 Cost of restrictions

The prohibited columns are defined by the code, in the `'forbidden'` list of the configuration, actual movement time, actual network pushback, blocking time, the target itself as a variable, and the runway actually used. Their exclusion incurs a measurable cost: 1 to 70 seconds depending on the airport for the runway actually used (project decision D-005); approximately 10 seconds – equivalent to 1.9 per cent RMSE – for the allocated position if it were removed; and 2.4 seconds for all six planning variables combined. These figures represent the premiums paid to comply with the rule, not defects in the deliverable.

3.4 Data tested and subsequently rejected

Two complete data sets were constructed, evaluated and discarded on the basis of the figures. External weather data—METAR reports collected from the Iowa Environmental Mesonet archive—proved to offer no measurable benefit in 2025 (as recorded in the decision); it has therefore been excluded from the solution and from this report. The traffic actually observed prior to the reference time – and therefore valid and testable – was rejected at the acceptance threshold for the reasons explained in section 3.1. These two rejections

are recorded in the repository's experiment log, which contains 32 entries, with the unsuccessful attempts listed alongside the successful ones.

4. Validation protocol

4.1 Checking for information leakage

To demonstrate that the model does not use the answer, we conducted a direct experiment, and here is exactly what it consists of. We took 50,000 starting points from the historical data. Using the code from the repository, we generated their 34 variables twice and their predictions twice: the first time using the files as provided by the competition organisers, and the second time by replacing the three calculated columns – take-off time, pushback time recorded by the network manager and holding time – with empty cells. If even a single one of the calculated variables were to read even a fragment of these columns, its values would change between the two runs, and the predictions would change with them. According to the results recorded on 10 October 2026, the matrix of variables is identical row by row, and the maximum difference between the two sets of predictions is 0.0 seconds across the 50,000 departures. In other words, these columns can be removed from the dataset; the model does not notice a thing, proving that it never needed them. The `contrôle_causal.py` script in the repository replays this experiment in a matter of seconds, and `test_clock.py` from the full project performs the equivalent automated check.

4.2 Time reserve: principle and use

The holdout set is a portion of the historical data that the model has never seen during training—in this case, the months of November and December 2025. The submitted model is trained on January to October only, as verified in the metadata of the frozen model and in the code, which excludes the holdout set from any training. We have set aside this data for practical use on three occasions. Firstly, to determine the model's size, with the number of trees being decided based on performance on months not used for training rather than on those used to train it. Secondly, to finalise the last adjustments to the recipe, without ever consulting the official score. Finally, to ensure the credibility of every internal figure cited in this report: when we report an RMSE during development, it is based on months the model had not been trained on – that is, on simulated future data – the only honest test whilst the actual future of 2026 was not yet available.

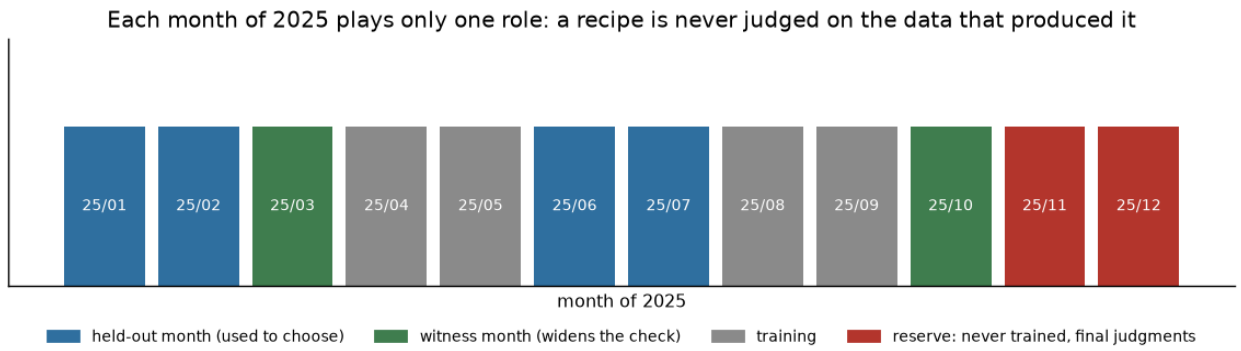


Figure 13: Protocol: training, control and reserve months

During the training phase, each month of 2025 has a fixed role that is known in advance: April, May, August and September (grey) contribute to the ongoing learning process; January, February, June and July are the selection months (blue): they are removed from the training in turn, one at a time, and the model lacking data for that month is the one to which the score is assigned; a prediction is therefore never assessed against a month it has learnt. March and October are excluded from the learning process in the same way; they function identically but do not determine the outcome; they broaden the verification (green). November and December form the reserve, the only group unknown to all models, reserved solely for final decisions (red). A month only records a model if that model has not learnt it.

4.3 Adoption threshold: determination and use

A change to the strategy is only adopted if it improves the RMSE over each of the four evaluation months by at least 1 per cent (project decision D-029). This threshold was set at the start of the campaign for a specific reason: gains below this level tended to fall, from one month to the next, within the range where our variants contradicted one another; sampling noise was sufficient to generate such results. It was applied at least three times, and on each occasion to resist the temptation. The average of four boosters trained on the same data gained 0.79 seconds in January and July, which was below the threshold and therefore rejected. Extending the training to November–December promised gains of 1.5 to 2.5 seconds but depleted the reserve and remained below the threshold; – rejected. The actual observed traffic showed an overall improvement of 0.32 per cent but worsened the results for January; this was also rejected. A threshold is of no use if it allows ideas to pass; it is useful when it rejects an idea that appears favourable.

4.4 Links between the 2025 measurements and the 2026 ranking

Our internal scores are calculated based on 2025, whilst the ranking assesses January and July 2026. To link the two without cheating, we measured a transfer factor based on the 670,407 arrival labels actually provided for those months, representing a 5-7 per cent increase. This enabled us to announce a range before each submission – for example, 631 to 644 seconds for the submitted version – rather than making adjustments after the event. We had set out the limits of the process in black and white, based on finish times; however,

its applicability to start times had not been verified. The official results confirmed this in our favour: the actual performance was around 1.4 % better than our local measurement, rather than the feared 5 to 7 per cent, as the recorded start times proved easier than the extrapolation from finish times had suggested. This finding did not alter the methodology in any way, and that is the key point.

5. Modelling and submitted solution

5.1 Single LightGBM model: 278 decision trees

The model selected is LightGBM, an open-source library of successive decision trees [1], where each tree corrects the errors of the previous ones and the final prediction is the sum of the leaves reached.

```
predicted_travel_time = max( 0; rounded to the nearest second of the sum across
the 278 trees in the reached leaf )
```

The input vector comprises the thirty-four parameters from Section 3.2, and the model contains 278 trees, a number determined by early stopping on months held out, never on the label of the month being evaluated. The lower bound of zero seconds eliminates impossible values, and no upper bound is applied, as clipping long running times cost up to 2.5 per cent of the RMSE during development; the tail therefore remains as it is.

Hyperparameter	Value	Hyperparameter	Value
objective	L2 regression, the official metric	learning_rate	0.05
num_leaves	96	min_data_in_leaf	200
feature_fraction and bagging	0.8 and 0.8	L2 regularisation	1.0
random seed	7	early termination	150 rounds of patience

The choice of a single model rather than a combination is based on the same principle as the adoption threshold; the combinations tested never exceeded a 1 per cent gain, and simplicity is a key factor in reproducibility. The `model.txt` file in the repository describes the 278 trees, which comprise 6010 lines annotated by the tool (2.6 Mo).

5.2 Model training: metadata configuration

The model is trained on data from January to October 2025, comprising 1,757,038 usable departures. November and December are excluded from the training process as time reserves, as confirmed by the model's metadata, where the list of training months is specified in plain text, and by the regeneration code which reconstructs this list whilst

excluding the reserve. No filter excludes the target queue during training, as this filter resulted in a 2.3 per cent cost during development (decision D-011 on the record).

5.3 Locked setting recipe

Date	Event
10 October 2026	`Taxipredict_v7new` package built from the frozen artefacts; cross-checks performed on the entire project; values identical line by line
11 October 2026	SHA-256 manifest regenerated, `b00028e4…` build file, final file for the four-month period `49f401e3….` The target is written in full 32-bit format as in the official templates, with the same values on every line as the project artefacts
11 October 2026, in the evening	`lighthearted-sandwich_v7.parquet` uploaded, renamed without altering any bytes, hash verified before submission

Regenerating the file oneself is a goal in itself. The `app/infere.py` script in the repository reloads the frozen model, reconstructs the variables from the fixed specification—including build flags to prevent any drift—applies the original post-processing, and refuses to write if a single compliance check fails.

6. Official results and critical analysis

6.1 Score used and forecast prior to submission

Metric	**Value**
Local RMSE for January and July 2025 (internal reference metric)	604.7 s
Range announced prior to the v7 submission	632 to 645 s
Official score for v7 submission	**595.3169 s**

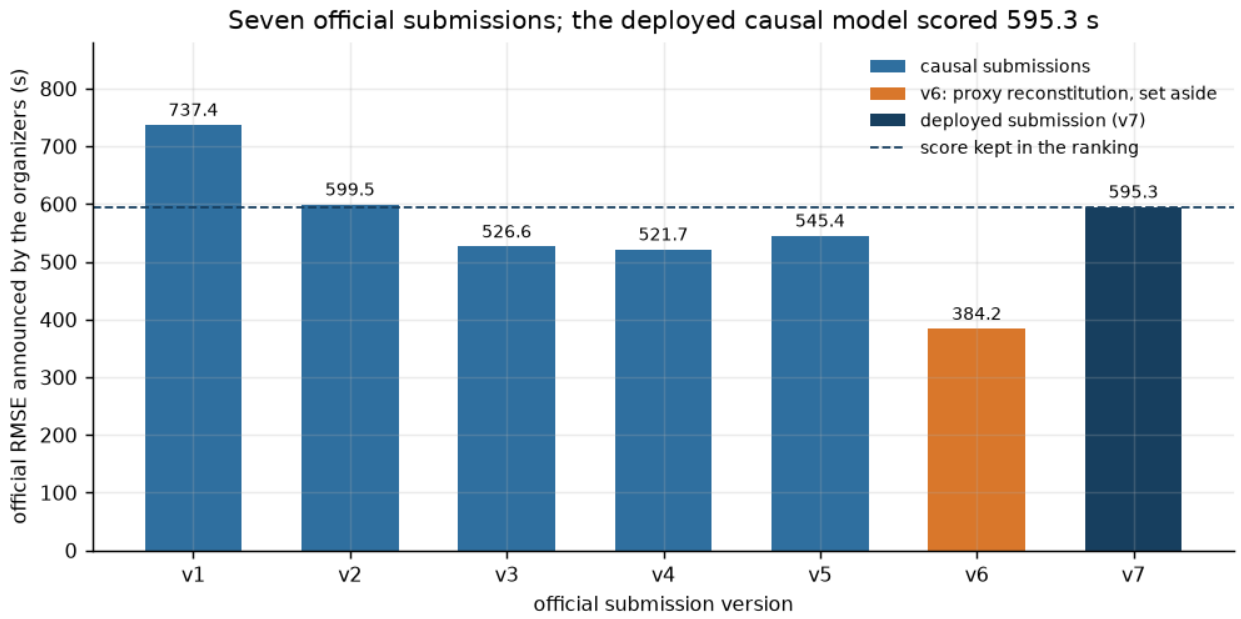


Figure 14: Official scores for the versions

The trajectory of our submissions tells our story: a rise to the low point of the v6 reconstruction (using the proxy), followed by a deliberate return to 595 s through the deliberate choice of v7. The dotted line marks the score used for the ranking, which is indeed that of the solution described in this report. The orange box shows the figure obtained without making a prediction; it serves here as a benchmark, not a target.

6.2 Breakdown by month and by airport

Over our four-month internal evaluation period, the RMSE for the submitted solution was 540 s in January, 555 s in February, 459 s in June and 650 s in July, giving an overall figure of 557 s and 605 s for the January–July period alone. July is consistently the most difficult, as indicated by the tail in Figure 7.

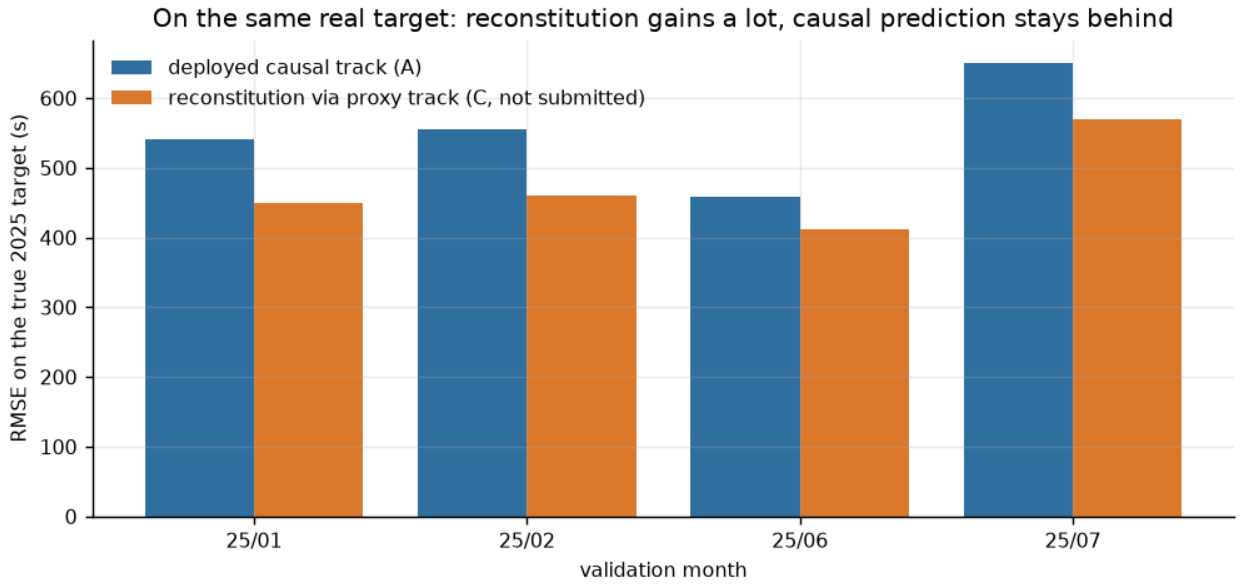


Figure 15: Causal RMSE vs. proxy reconstruction

On the actual 2025 target, the proxy reconstruction track gains around 14 per cent across the board, 480 seconds compared with 557 seconds overall. This is the most accurate internal discrepancy between the two questions: how long will it run and how long has it run. The official discrepancy of 211 seconds corresponds to this in terms of direction and, roughly, in relative magnitude.

By airport, the RMSE ranges from 242 s in Madrid and 246 s in Barcelona to 546 s at Paris CDG and 1,510 s at Rome Fiumicino. Rome accounts for a high concentration of error due to its population of questionable routes identified in section 2.5, on which even the most naive proxy outperformed our model – an admission made without shame, since no one can predict measurement error.

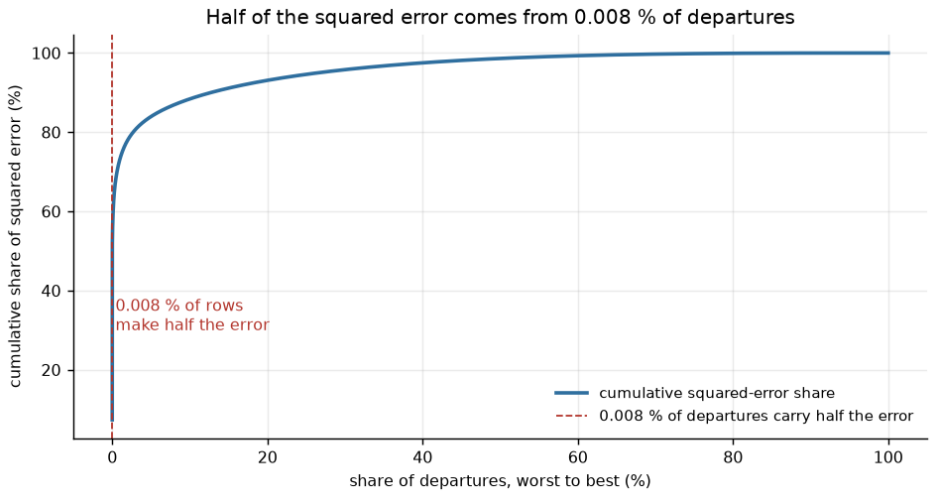


Figure 16: Concentration of error

Approximately one route in a thousand accounts for half the square of the total error. This figure explains why no global calibration trick shifts the score, and why we state that the remaining improvement depends on the quality of the data in Rome, not on a larger model.

6.3 Rejected variants

Variant	Internal measurement	Verdict and reason
Average of 4 boosters on the same data and variables	0.79 s gain over January–July, interval [0.9974; 0.99997]	Rejected, below the D-029 threshold
Training extended to November–December	Estimated gain of 1.5 to 2.5 s	Rejected, below the threshold and unverifiable due to lack of reserve
Six methods for handling outliers (clipping, filtering, robust loss)	losses of 0.41 per cent to 4.79 per cent per month	loss across all four runs
Traffic actually observed prior to the current moment (`refattente-r1`)	−0.32 per cent overall; January +0.87 per cent	rejected (D-037), below the threshold and out of season
External METAR weather data (Iowa Environmental Mesonet)	measurable zero gain in 2025	rejected, and no external data is included in the repository

A rejected variant is a decision; it deserves the same documentation as an accepted variant. The artefacts from the rejected explorations remain in the repository and can be consulted; they document a research process, not a repository.

7. Risk of data leakage: methodological dilemma and limitations of our hypothesis

7.1 Two interpretations of the verb ‘predict’

The regulations use **the term ‘predict’** but specify neither a specific point in time nor a list of columns. Our rule is that the absence of a prohibition does not constitute authorisation. The other interpretation, which is permissible in the strict sense of the text, utilises the traces left by the theft within the files themselves—a possibility opened up as early as section 1.4. Let us quantify the discrepancy; it sums up the entire discussion.

Interpretation	What it actually reads	RMSE for January and July
Pure reconstruction (our v6)	take-off achieved minus network boost	384.16 s
Causal prediction (our submitted v7)	only the pre-thrust phase	**595.32 s**

We are not making any accusations regarding other teams' scores. We are simply observing that the structure of the data makes certain performance results compatible with two very different strategies, and that an outside observer cannot distinguish between them. This is the problem we are highlighting, backed up by the figures.

7.2 Our trajectory, v1 to v7

Version	Official score	Track	What happened, and the lesson
v1	737	prediction	first complete causal model; it proved the feasibility. Lesson: a solid start is not a ceiling
v2	599	prediction	better planning variables and planned traffic. Lesson: in causal modelling too, variable engineering pays off
v3	526	hybrid with indirect proxy	first blend close to take-off. Lesson learnt: the gain was real but was already taking the back door
v4	521	hybrid	refinement of the same approach. Lesson: one can make progress for a long time without realising that one is tackling a different problem
v5	545	hybrid	a 24-second delay following an internal prediction of +150 seconds. Lesson learnt in a score read from a reconstructed label only predicts the true value if its underlying correlation is stable, and it is not
v6	384	pure reconstruction	the proxy alone, with almost no model, the team's best result. The lesson is that it was precisely because this figure was easy to obtain that it was worthless as a prediction
v7	**595**	**submitted**	**back to causal modelling with causality control. Result: +211 on the official score**

In early October, whilst auditing the columns, two observations became clear: the target is a direct reflection of historical data, and the test revealed that the 500 barriers had been broken. This led to the real team debate, which was not technical in nature: does the competition assess a prediction or a reconstruction? We decided to judge ourselves based

on the question asked. The three most lessons from this journey are: the slippery slope of hybrids is difficult to see from the inside without an explicit rule of the moment; a local indicator can lie in good faith; and an easy score can be the best argument against its own use.

7.3 Causal assumptions: conditions of validity and consequences of their violation

Our consistency check (section 4.1) proves one thing: the model does not depend on the actual columns, with a deviation of zero over 50,000 departures. It does not prove everything. The model reads three hours' worth of rescheduling planned by the network and the assigned post, assuming this information was recorded before the train's departure, because its operational role is to anticipate. A static snapshot of the files does not allow us to verify the exact moment at which they were entered. This is a design assumption, not an established fact, and we state this clearly. Nevertheless, we verify it as far as possible, and we validate it through two analyses carried out on the two million departures in 2025.

There is no evidence of a late entry. The last known push time matches the actual push time exactly in only 0.4 per cent of cases; on a median basis, it is 251 seconds earlier, and on average, six minutes earlier. Its statistical profile is that of a planning field. By way of comparison, the value recorded by the network – which we exclude from the causal analysis – is within a few tens of seconds of the actual time, with a median of −51 s. It should be noted, without concealing it, that around a quarter of the lines have a last known value that is later than the actual push time, which suggests late revisions to the network rather than systematic copying.

The worst-case scenario has been quantified.

Two ablations quantify the risk carried by these assumptions. Both show small effects. First, we remove only the two variables built from the last known push time: its deviation from schedule and its absence flag. This reduces the submitted specification from 34 parameters to 32. The cost is 0.2 seconds of development RMSE, or 0.04 per cent. Month by month, the changes are +0.5, −0.8, +0.9 and +0.1 seconds. This is indistinguishable from no effect at all. We also remove the whole planned-network block. This comprises all six variables derived from the estimated, initial and last-known push times, together with their absence flags. The parameter count falls from 34 to 28. The overall cost is 2.4 seconds, or 0.4 per cent. The monthly changes are +2.0 s in January, +3.1 s in February, +3.4 s in June and +1.4 s in July. The assigned stand is the second assumption we rely on. Removing it reduces the parameter count from 34 to 30. The cost is 10.5 seconds, or 1.9 per cent. These penalties were measured separately and are not additive. Even taken together, however, they bound our total exposure to these design assumptions at well under 15 seconds of development RMSE. By comparison, the gap between our submitted causal score of 595.3 s and the 384.2 s obtained by pure reconstruction on the official metric is 211 seconds. In other words, even if every one of these assumptions were to fail simultaneously, the submitted solution would remain far from the level of a method that reads the answer in the inputs. We report a

difference between two questions: how long a flight will taxi versus how long it did taxi. That difference stays well outside the margin our own hypotheses could explain.

7.4 Recommendations to the organizers

Three recommendations emerge from our experience of the competition. They are intended to benefit the competition, whatever happens next.

1. **Specify the prediction time** in the rules; for example, no information generated after the start of the run. A single sentence is enough to make the metric unambiguous.
2. **Publish the list of eligible columns** and their availability at that point in time. At present, two teams may interpret the same text as imposing conflicting requirements.
3. **Neutralize the algebraic gate for future editions**, by rounding the movement time to the nearest minute, shifting it by a constant, or masking it as the lock time.

8. Control via a free, local AI

8.1 The tooled chain

We have worked with the AI, not under the AI. We have called this approach A/(2L-R), standing for Approach/(**Free**, **Local** and **Reproducible)**.

```
team (questions, rules, decisions)


    OpenCode, agent-based framework, reads and writes files,
    executes, searches

    Arbor[], autonomous search orchestration
    Persistent idea tree, isolated executors on branches dedicated
    Git branches, hypothesis-driven proof folder,; merged into the
    main branch only if the gain is confirmed


    local models: DeepSeek V4 Flash (0731); Qwen3.8-Flash-Next
```

OpenCode breaks the problem down into tasks and sets agents to work on code generation, data audits and summarization. Arbor AI [3] structures the exploration; each idea is a node carrying a falsifiable hypothesis, its score and its verdict, developed in an isolated copy of the project and merged only if the gain is confirmed over the months spent outside of training. The state of the research persisted and visualized; it is not a line of reasoning that is lost or recovered with the session's memory. A log of experiments and a ban on adjusting hyperparameters without criteria form the basis of the chain's default operation.

8.2 Why these two models, and why run them locally

Feature	DeepSeek V4 Flash (0731) [4]	Qwen 3.8 Flash-Next [5]
Architecture	Mixture of Experts (MoE)	Mixture of Experts (MoE)
Context window	greater than 200,000 tokens	greater than 200,000 tokens
Decoding parameters	default values	default values
Speed measured on our equipment	of the same order of magnitude as Qwen	30 to 40 tokens/s
Runtime hardware	Mac Studio, NVIDIA DGX Spark (GB10 same chip), NVIDIA graphics cards	

With each token generated, only a fraction of the network's weights is activated, whilst the irrelevant experts remain dormant. For equivalent quality on agent task, reading, deciding, editing, verifying, the throughput is significantly higher, and these are long loops that need to be fed, involving hundreds of iterations per experiment. A context window of over 200,000 tokens allows us to keep an entire experiment dataset in view.

We prefer models that are slightly less powerful but open-source, with freely configurable weights that run on our own hardware, rather than commercial black-box solutions. Our approach must be understandable, auditable and reproducible without relying on a third-party service or revealing our working data. As any customization of the decoding process is a source of irreproducibility, hyperparameters and the like remain at their default values; seeds are recorded when relevant, and the method is repeated. The throughput figures quoted are measured in-house in a real-world agent-based inference configuration rather than taken from commercial documentation; their inferiority compared to a commercial service is precisely the trade-off we accept in order to retain control.

8.3 The balance of things: what the AI did not decide

Structural decisions bear a reference to human decision-making recorded in the project documents: the prediction time set at deployment; the abandonment of the reconstruction track after v6; the D-029 adoption threshold; the rejection of the average booster score; and the prohibition on adjusting based on ranking scores. The AI explored, quantified, argued and codified, including producing the analyses that revealed the data leak to us. The final decision remained a human one. It is this division of responsibilities that ensures the original solution clause is genuinely adhered to and documented, rather than circumvented. The chain logs published in the repository have been filtered – with no local paths, keys or sensitive content, retaining only those extracts showing an idea emerging, being challenged, assessed and then judged.

9. Reproducibility, acknowledged limitations and conclusion

9.1 Obtaining the data, not redistributed

The organiser's files are not republished in the repository, in order to comply with their terms of use. Once the keys have been obtained (OpenSky account, 'Using MinIO client' section of the Data page), the relevant files have the following hashes, as recorded on our copies.

File	Role	SHA-256 hash
`final_ranking.parquet`	variables and finish line labels for January to July	`4b0692a7…`
`submitting.parquet`	ranking template, 344,841 lines	to be noted in the downloaded file
`final_submitting.parquet`	final response template, 670,790 lines	to be noted in the downloaded file

9.2 Step-by-step reproduction

```
# Environment: on macOS, install libomp if LightGBM fails to load
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Regenerate the submission from the frozen model. The script fails and
outputs nothing if any check fails.
python app/infere.py --famille A          # both templates, causal track

# replayable evidence
python app/controle_causal.py             # 50,000 runs, expected deviation
0.0 s
shasum -a 256 -c MANIFEST.sha256          # files and models, hash-by-hash
```

Three properties of these commands act as safeguards. The model specification is fixed; the variable construction flags come from `spec.json`, so no drift is possible. Writing is done in 32-bit integers in accordance with the official templates. And any failed check terminates the output with an error code; nothing is submitted blindly.

9.3 What was actually submitted

File	Lines	Footprint
`lighthearted-sandwich_v7.parquet`, January and July, score 595.3169	344,841	`b00028e4…`
`lighthearted-sandwich_final.parquet`, single response over four months	670,790	`49f401e3…`

A note on equivalence should be read before making any comparison with the project's artefacts; the target is written in full as 32-bit values in these files and as 64-bit floating-point values in the project, with the same values on every line, verified by maximum difference being zero, but with different bytes.

9.4 Our limitations, without compromise

- **Design assumptions.** Network hours and the position are assumed to be recorded prior to pushback; the audit in section 7.3 makes this assumption plausible and caps the cost at 0.4 per cent in the worst-case scenario, without treating it as a fact.
- **Expected ground distances.** These are derived from the 2025 taxiing runs, and thus from historical experience rather than information about the predicted flight. This is permissible under our protocol, but it is important to be aware of this.
- **Local factor affecting classification.** Measured on arrivals, its application to departures proved to be an overestimate, with an actual figure of around -1.4% compared with the announced +5 to +7 per cent. The sign was favourable to us, but the estimation method still needs improving.
- **The queue at Rome Fiumicino.** On certain routes, the simplest proxy outperformed our model, and we do not claim to be able to explain this. These routes, with clearly erroneous targets and a hold time equal to the scheduled time in a proportion twenty times higher than normal, will remain unpredictable for any honest model. Their inclusion in the test is a lottery that no one can rig from the inside.

Given more time, we would run the protocol again as soon as the January to July 2026 data is published as historical data – the first truly 'out-of-the-future' month. We would calibrate the tails rather than clip them, as rejecting the upper limit carries a measurable cost that calls for explicit modelling of the probability of extremes. And we would reopen the weather data, which is neutral on average for 2025 but may not be for individual de-icing episodes.

9.5 Conclusion

During the period covering our participation, we achieved a better score than the one we have decided to submit, and that is our result. The discovery of a loophole in the data, where the target can be read almost verbatim, posed a question we had to resolve: **what do we promise when we say we are predicting?** We responded with rules of conduct for modelling:

- a specific prediction time chosen and adhered to,

- a reproducible proof of the absence of leakage.

We compared the two approaches (reconstruction and causal), rejecting variants because they fell below the threshold rather than adopting them simply because they yielded favourable results (in terms of score).

We advocate the use of open-source, locally deployed AI implemented by us, with decisions made by the team whilst the tool carried them out. Our score of 595.32 seconds reflects the value of these constraints.

An open question for the community: if a future edition were to set a fixed prediction time, how many teams would revert to a causal approach, and would the rankings shift by more than the 211-second gap we conceded by adopting the causal approach? We are publishing these figures so that this question can be asked on the basis of data rather than mere intentions.

Annex — Open-source stack, AI tooling and licence compliance

A.1 Python stack (versions pinned as recorded on 13 October 2026)

Package	Version	Licence	Role in the project
Python	`3.13.14`	PSF	language runtime
Numpy [7]	`2.5.3`	BSD-3-Clause	array computation
Pandas [8]	`3.0.6`	BSD-3-Clause	tabular data
pyarrow	`26.0.0`	Apache-2.0	Parquet I/O (official data files)
LightGBM [1]	`4.7.0`	MIT	gradient-boosted trees — the model itself
scikit-learn [9]	`1.9.1`	BSD-3-Clause	metrics and utilities
Matplotlib [10]	`3.11.2`	matplotlib licence (PSF-style)	all figures in this report
pytest	`9.1.1`	MIT	test suite (clock test, calibration guards)

The inference package ships these pins verbatim in `requirements.txt`; the model specification and construction flags are frozen in `models/causal/spec.json`.

A.2 AI orchestration tooling (all open source)

Tool	Role here	Licence / reference
OpenCode [2]	agentic harness: task decomposition, file editing, shell execution, audits and drafting	MIT — https://opencode.ai , https://github.com/anomalyco/opencode
Arbor [3]	autonomous research orchestration: persistent idea tree, isolated executors on Git branches, guarded merges	Apache-2.0 — J. Jin et al., *Toward Generalist Autonomous Research via Hypothesis-Tree Refinement*, arXiv 2026; https://github.com/RUC-NLPIR/Arbor (PyPI `arbor-agent`)

Tool	Role here	Licence / reference
Unsloth Studio (local)	serves the open-weight models below through a local API on our own hardware	open source — https://github.com/unsloth ai

A.3 Language models (open weights, run locally)

Model	Use	Licence / reference
DeepSeek-V4-Flash-0731 [4]	agentic coding and orchestration loops	MIT — https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-0731 ; technical report arXiv:2606.19348
Qwen3.8-Flash-Next [5]	agentic coding, drafting, this report's first English pass	open weights (Qwen team) — https://huggingface.co/Qwen

A.4 License compliance statement

The PRC Data Challenge 2026 eligibility rules require open data, a **public GitHub repository under GNU GPLv3**, documentation sufficient to reproduce the results, and an original solution [6]. We comply:

1. all project code V1 pipeline, V2 campaigns the internal campaign series that preceded it, figure generation, this report's annex builder is released under **GNU GPLv3** at **[repository URL to be added before submission]** ;
2. no external dataset is used in the submitted solution (the METAR [12] experiment was measured and rejected, and its data is not part of the repository);
3. reproduction requires four commands (section 9.2) and fails loudly if any compliance check breaks;
4. the AI tooling itself is open source (B.2) and the language models carry open-weight licences (B.3), so the full chain — data to prediction — is auditable without commercial black boxes.

References

1. G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*, NeurIPS 30 (2017). https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html — implementation: https://github.com/microsoft/LightGBM (MIT).
2. Anomaly, *OpenCode — the open source AI coding agent*. https://opencode.ai — https://github.com/anomalyco/opencode (MIT).
3. J. Jin, Y. Hu, K. Qiu, Q. Dai, C. Luo, et al., *Toward Generalist Autonomous Research via Hypothesis-Tree Refinement*, arXiv (2026). https://github.com/RUC-NLPIR/Arbor (Apache-2.0), PyPI: `arbor-agent`.
4. DeepSeek-AI, *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*, arXiv:2606.19348 (2026); model card https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-0731 (MIT).
5. Qwen team, Alibaba Group, *Qwen3.8* open-weight models. https://huggingface.co/Qwen.
6. EUROCONTROL Performance Review Commission, *PRC Data Challenge 2026 — On predicting taxi-out time of flights*. https://ansperformance.eu/study/data-challenge/dc2026 (data, rules and eligibility pages).
7. C. R. Harris et al., *Array programming with NumPy*, Nature 585, 357-362 (2020). https://github.com/numpy/numpy (BSD-3-Clause).
8. W. McKinney, *Data Structures for Statistical Computing in Python*, Proc. Python in Science 2010. https://github.com/pandas-dev/pandas (BSD-3-Clause).
9. F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12 (2011). https://github.com/scikit-learn/scikit-learn (BSD-3-Clause).
10. J. D. Hunter, *Matplotlib: A 2D Graphics Environment*, Computing in Science & Engineering 9(3), 90-95 (2007). https://github.com/matplotlib/matplotlib.
11. Apache Arrow project, *Apache Parquet / PyArrow*. https://arrow.apache.org (Apache-2.0).
12. Iowa Environmental Mesonet, Iowa State University — METAR archive used in the rejected weather experiment. https://mesonet.agron.iastate.edu.